INSIGHTWELLNESS AI

Milestone 4 — From feedback to a production-grade ML system

INFO9023 Machine Learning Systems Design · ULiège · Spring 2026

Team: Arsanov Ramzan · Ntakirutimana Renaux

InsightWellness AI in one minute

The product

- Early obesity-risk detection: predicts one of 7 classes (Insufficient_Weight → Obesity_Type_III)
- Every prediction is explained
- A chat assistant answers follow-up questions, grounded in the model and a curated knowledge base

The system

- HistGradientBoostingClassifier (scikit-learn), trained and tuned by a Vertex AI pipeline
- Flask API + Streamlit dashboard, deployed on Cloud Run
- Multi-agent assistant (Agno + Gemini 2.5 Flash) behind POST /chat

MS3 feedback → what we changed

MS3 feedback	What we changed in MS4
“Not really a good CI/CD … doesn't check anything”	Gated pipeline: tests before every deploy, health checks, branch protection
“It just rebuilds everything on every push”	Path filters: only the changed service is rebuilt and redeployed
“Why two backends? No logic for splitting it in two”	Agent service merged into the main API as POST /chat; agent-api deleted
“Project ID shouldn't be directly in a code script”	All GCP config via env vars + GitHub variables; zero identifiers in code

Codebase redesign

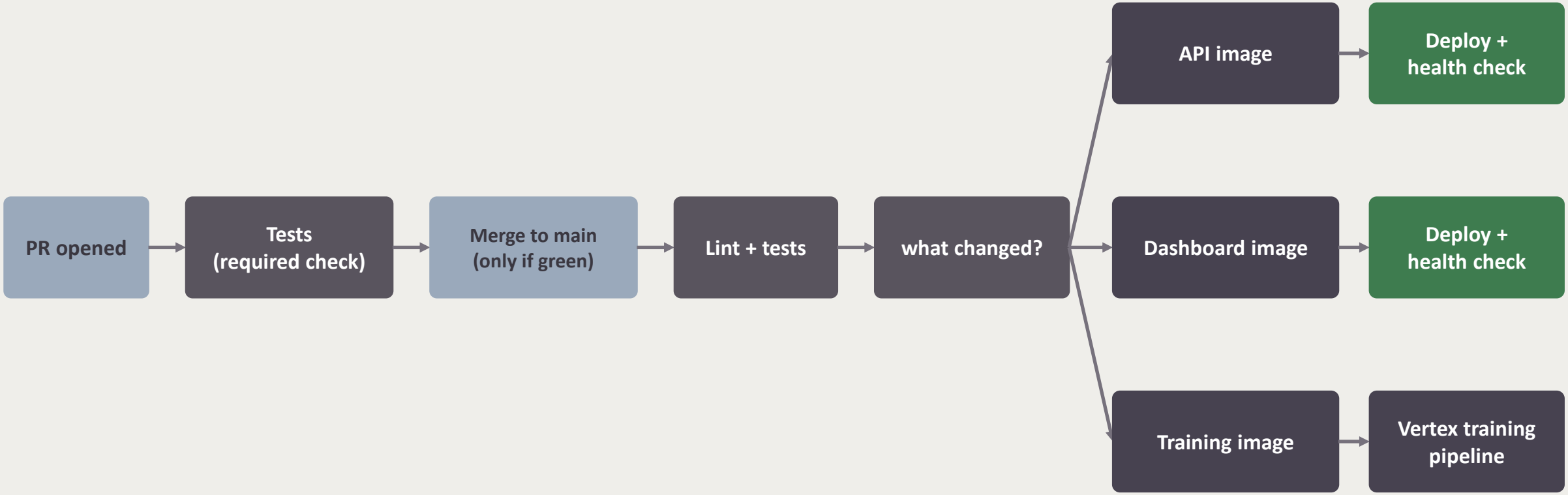
Before (MS3)

```
insightwellness_ai/
├─ api/app.py
├─ preprocess.py
├─ model_selection.py
├─ training.py
├─ evaluation.py
├─ streamlit_app.py
└─ agents/
```

After (MS4)

```
insightwellness_ai/
├─ config.py
├─ api/
├─ agents/
├─ pipeline/
└─ dashboard/
vertex/
tests/
```

CI/CD rebuilt: gated, filtered, verified



The gate is real: tests + branch protection

- 32 tests (offline, +/- 2 seconds, no cloud needed):
  - API behavior
  - preprocessing
  - pipeline, API and dashboard must agree on features and labels
- Branch protection: the only way into main is a PR with a green check and main is the only thing that deploys

One backend: agents merged into the API

Before (MS3)

2 services · 2 Dockerfiles · 2 deploy workflows

After (MS4)

1 service · 1 image · 1 pipeline · agents lazy-load on first /chat

- The chatbot is a feature of the model API, not a second system — deleted: Dockerfile.agent, agent_api.yml, agents/app.py
- The LLM orchestrates: predictions come from the model, explanations from SHAP and health facts from the knowledge base

Load test: found and fixed a real bottleneck

50 concurrent clients on /predict

Before

11.4 s

median latency · 2.9 req/s · 19% timeouts
never scaled past one instance

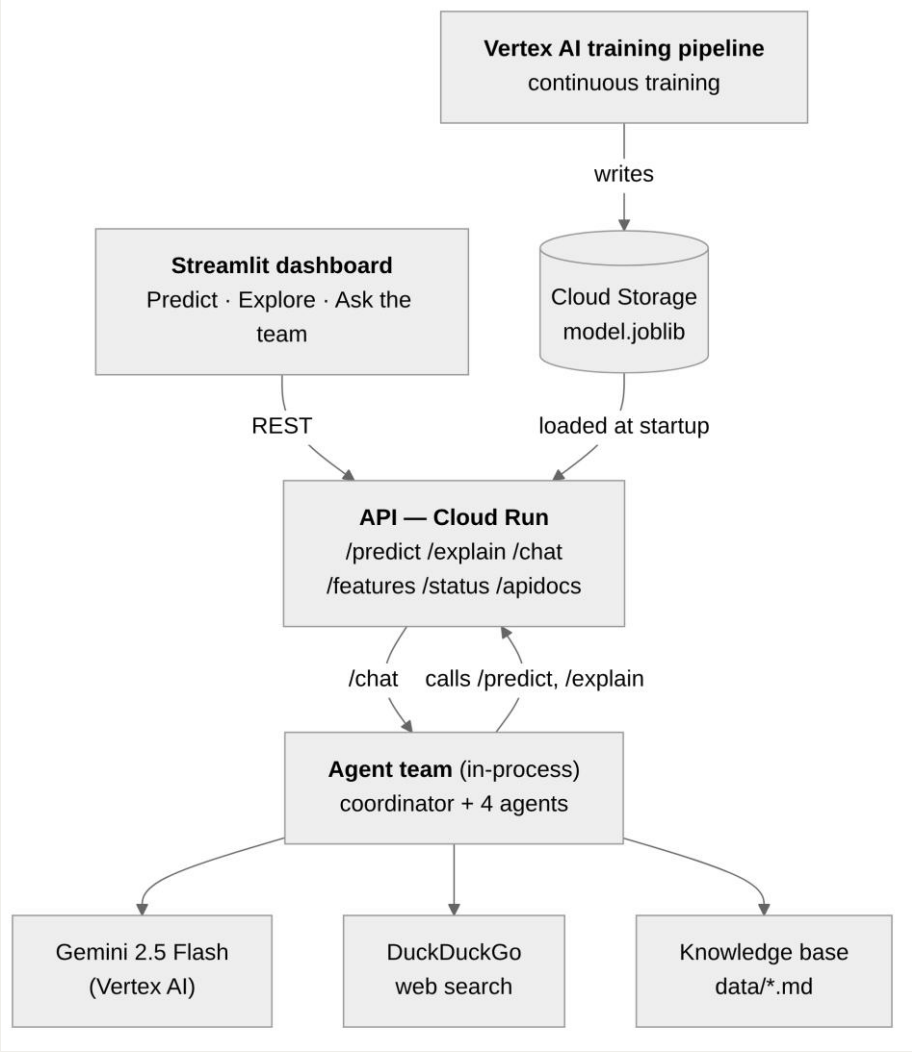
After

1.05 s

median latency · 14 req/s ·0% errors
autoscaled 1 → 5 instances

- Cause: Cloud Run concurrency set to 50 vs gunicorn capacity at 8: requests queued per instance instead of triggering scale-out; invisible to all functional tests

System architecture



Live demo

Dashboard	streamlit-app-nolzra3zua-ew.a.run.app
API docs	insightwellness-api-nolzra3zua-ew.a.run.app/apidocs
Code	github.com/RenauxNt/MLSD-InsightWelness-AI

Thank you.